

1. Constats

Bloquants (résultat faux pour le client)

#	Localisation	Constat	Principe violé
B1	engine/Checkout.java:30-40	L'offre « 2 achetées, la 3 ^e offerte » n'offre qu'un seul article , quel que soit le nombre de boissons. Reproduit : 6 boissons à 3 € → remise de 3 € au lieu de 6 €. La règle métier (un article offert par tranche de trois) n'est pas implémentée.	Exactitude métier (demande 3)
B2	engine/Checkout.java:55-80 et :112	La TVA est calculée sur le montant brut , avant remise, alors que le total HT affiché est après remise. Reproduit : pain à 60 € avec remise 10 % → HT 54 €, TVA affichée 3,30 € (5,5 % de 60) au lieu de 2,97 €. Le ticket est incohérent : la TVA ne correspond pas au HT qu'il affiche.	Exactitude métier (demande 4)
B3	engine/Checkout.java:102	Quand la remise fidélité est plafonnée au sous-total (:8-10), la formule $((int) \text{ bestDiscount} / 5) * 100$ retire 0 point . Reproduit : panier à 3 €, 100 points → remise de 3 € offerte, aucun point débité. Le client peut répéter l'opération indéfiniment.	Exactitude métier (demande 5)

Majeurs (conception, fragilité)

#	Localisation	Constat	Principe violé
M1	engine/Checkout.java:83-124	receipt() calcule la remise, calcule la TVA, débite et crédite la carte de fidélité , et affiche. Une méthode d'affichage a des effets de bord : appeler receipt() deux fois sur le même panier crédite les points deux fois.	SRP, séparation commande/requête
M2	engine/Checkout.java:94-105	Pour savoir quelle remise a gagné, le code recalcule les trois remises et compare des double avec ==. La décision prise dans getBestDiscount() (:45-52) est perdue puis devinée à nouveau.	DRY, fragilité des comparaisons flottantes
M3	engine/Checkout.java (classe entière)	Aucun objet ticket n'existe côté Java : receipt() écrit directement dans System.out et ne renvoie rien. Le domaine du sujet (Checkout produit le ticket) n'est pas respecté, et le ticket ne peut être ni testé, ni transmis au client TypeScript.	Séparation domaine/présentation
M4	engine/Cart.java:16-18 et :21-23	Le constructeur garde la référence de la liste reçue et getCartLines() renvoie la liste interne modifiable. Reproduit : une ligne ajoutée à la liste d'origine après création apparaît dans le panier. Main.java:12-29 en est victime : totalCart et totalCart2 partagent la même liste.	Encapsulation, copie défensive
M5	engine/Checkout.java:118-120	Montants en double affichés sans arrondi. Exécution de Main : TVA(20%) : 0.30000000000000004\$.	Représentation monétaire (BigDecimal ou centimes en long)
M6	engine/Checkout.java:55-80	Deux méthodes quasi identiques (getTVAFood, getTVAOther) avec taux codés en dur. Ajouter un taux impose une nouvelle méthode et la modification de receipt().	DRY, OCP
M7	Racine du dépôt	Pas de docs/, pas de README.md (livrable 7), pas d'outil de build (ni pom.xml ni Gradle), classes dans le package par défaut. Aucune instruction pour compiler ou exécuter.	Structure imposée par le sujet
M8	Dépôt entier	Aucun test automatisé. La « zéro régression » exigée au refactoring ne pourra pas être prouvée autrement qu'à l'œil.	Testabilité
M9	demo/receipt.ts:3-6	L'interface Receipt ne contient que lines et total : ni HT, ni TVA par taux, ni remise retenue, ni points. Elle n'est d'ailleurs jamais utilisée dans index.ts, qui recalcule son total sans remise ni TVA (:39-47). Les types ne décrivent pas réellement le ticket.	Exigence de typage du sujet

Mineurs

#	Localisation	Constat	Principe violé
m1	engine/CartLine.java:4 et engine/Product.java:2	Aucune validation : quantité nulle ou négative, prix négatif acceptés. Un compact constructor de record suffirait.	Programmation défensive
m2	engine/Fidelity.java:18-20	spendPoints peut rendre le solde négatif.	Invariant de l'objet
m3	engine/Checkout.java:16-17	Seuil 50.0 et taux 0.10 en nombres magiques.	Lisibilité

#	Localisation	Constat	Principe violé
m4	engine/Checkout.java:88-122	Ticket affiché en anglais et en dollars (\$) pour une chaîne de magasins française. Le sujet réserve le français à l'interface.	Consigne du sujet
m5	engine/Checkout.java:114	Les points sont calculés sur le HT après remise, alors que « un euro dépensé » désigne plutôt le montant payé (TTC). Choix à justifier.	Ambiguïté métier non documentée
m6	engine/LambdaServer.java	Serveur HTTP hors du socle « POO pure » du Temps 1. Il ne fait que renvoyer un accusé de réception et n'appelle pas le moteur. :43 utilise getBytes("UTF-8") au lieu de StandardCharsets.UTF_8.	Périmètre du sujet, YAGNI
m7	engine/CartLine.java:1	Import java.util.List inutilisé.	Propreté
m8	.idea/, historique v2	Dossier IDE versionné. Classes compilées (out/production) committées jusqu'en v2.	Hygiène du dépôt
m9	Tag v5	Commit unique mêlant nouvelle fonctionnalité et « rewriting of methods ». La reprise de code se confond avec l'ajout de la demande 5.	Commits atomiques

2. Points de rupture

Ajout d'une nouvelle remise. Il faut toucher trois endroits de Checkout : une nouvelle méthode, `getBestDiscount()` (:45-52) et la chaîne `if/else` de `receipt()` (:96-105). Si deux remises valent le même montant, c'est l'ordre des `if` qui décide du libellé affiché et de l'éventuel débit de points, pas une règle métier.

Ajout d'un taux ou d'une catégorie. Une nouvelle méthode `getTVAXxx()` et la modification de `receipt()` (:112, :118-119). L'association catégorie → taux n'existe nulle part comme donnée.

Changement de format du ticket (JSON pour le client TypeScript, impression, e-mail). C'est impossible sans réécrire `receipt()`, puisque calcul, effets de bord et affichage y sont mêlés et qu'aucun objet ticket n'est produit.

Règles de cumul. Le non-cumul est codé en dur par `Math.max`. Toute exception (« la fidélité se cumule avec l'offre boissons ») oblige à réécrire la logique de sélection.

3. Sixième demande client

*Nouvelle promotion sur l'alimentaire : tout produit alimentaire acheté en 5 unités ou plus bénéficie de 20 % de remise sur sa ligne.
Comme les autres, elle ne se cumule pas : on retient toujours la remise la plus avantageuse pour le client.
Le ticket indique le nom de la remise retenue et son montant.*

Pourquoi cette demande : dans un code où chaque remise est une stratégie indépendante, elle s'ajoute par une nouvelle classe, sans toucher aux autres. Ici, elle impose de modifier les trois endroits décrits au premier point de rupture, et de rajouter une branche à la détection par égalité de `double` (M2). Elle respecte le cadre : règle de remise, moins de cinq lignes, faisable en deux heures dans un code bien conçu, aucune des cinq demandes précédentes remise en cause.